MetropoLive

Group 17 - Sunsets

Yuginda Ranawaka, Tyrese Temple, Sumin Eun, Jacob Yan

https://github.com/CMPT-276-SUMMER-2025/final-project-17-sunsets

https://metropolive.onrender.com/

Analysis of the project's success in meeting the user needs and solving the problem (Include feedback from classmates):

Our project is successful in meeting the user needs and solving the problem. The user's needs include making connections, being able to explore local events, and being able to fulfill these needs in a convenient and simple way. The problem is it is harder than ever for people to socialize and find third places. Our application solves this problem and fulfills those needs by providing an application that finds nearby events based on user's preference and also providing information about the event along with a route to said event. The success is confirmed by the feedback we received from our peers. Many of them found our application intuitive and easy to use, stating "The website is intuitive to use and pleasing to look at" or "The website is very straight forward to use, no confusion". Many of them also did not have any suggestions because our application already works as intended.

Analysis of the project's SDLC model and how it was used in the project (Include any changes made to the model):

We used the agile kanban SDLC model, since it is the simplest model shown in class and the easiest to understand. Github already has a kanban model on projects, making it convenient for us to utilize the model since everything from issues to the code are already connected. The way we utilized the model for our project is by making issues, such as bugs, enhancements, and documentation, then connecting issues to its corresponding project. We had three kanban projects, M0 - Resubmission, M1 - Report, and MetropoLive build. We would update the progress of our issues in real time so others would be able to see what was being worked on and by who. Some issues were, "Event cards", "Logo", "Event Routing & Polyline Display", etc. The issues also clearly showed us what needed to be worked on. It also helped assign people to certain tasks since we could all choose which issues to tackle. Throughout using the model, no changes were made. The kanban system was sufficient enough to fulfill all our needs.

Detailed description of the features implemented for each API (Include any changes made):

Ticketmaster Discovery API:

Core Features:

- **Event Search:** Search for events in real-time. Data returned is provided by Ticketmaster's Discovery API.

- **Location-based Search:** Enter a city to find nearby events (longitude/latitude-based search is being depreciated in Discovery API, so targeting a user's immediate location is done at city-level and not coordinates).
- **Event Details:** Important event information including dates, locations, a link to purchase tickets, and descriptions are included in the cards. Note that not all events provide information such as descriptions, for which an “unavailable” notice is displayed.
- **Ticket Links:** Click the "Buy Tickets" button to purchase tickets directly from the event's page (Ticketmaster).

User Workflow:

- Enter a city name in the location field, or a particular address.
- Optionally add keywords to filter events (e.g., "concert", "food").
- Provide input for each of the search preferences to further filter results (e.g., radius, date, price, # of events). Like the keyword filter, each of the input boxes that take numeric values have associated defaults and are all optional to use.
- Click "Search Events" to return events in card form.
- Click on any event card to expand and see more details such as price, a link to its respective ticket page, or navigation.
- Use the “Back to Events” button to return to the search form.

API Integration

As mentioned earlier, this application uses the Ticketmaster Discovery API to fetch real-time event data. This was chosen as Ticketmaster provides an event discovery service that doesn't first require registration and later approval in order to use. Meetup GrapQL API unfortunately does, and I hadn't found this information within the development docs at the time of researching this API. The one core API feature that MetropoLive uses is an event search functionality that is made possible with the following Discovery API endpoints:

- **Event Search - events.json Endpoint:** users can search for events by city-level location in combination with several search preferences. Chosen query parameters build out the subsequent Discovery API call, and return relevant event options to the user.
    - **Search Preferences:**
        - City: primary location of event search; input box found in site header.
        - Category: keyword that describes an interest (e.g., music, food, sports).

        - Search Radius: the radius of search for events within a given location.
        - From/To: the interval of time for which to filter events. This is particularly useful for those with busy and continually shifting schedules.
        - Number of Events: displayed number of events within the content panel. Minimum is by default 1, and the max is 50.
- **Get Event Details - <endpoint here>:** since Event Search doesn't include a request parameter for ticket prices, a call to the Get Event Details endpoint is made using each returned event's unique ID. These secondary calls return price ranges in array objects, and the events that fall within the user's price range are displayed.
    - **Search Preferences:**
        - Min/Max Price: price limits for event tickets, with currency is set to CAD.

Google Maps JavaScript API:

Core Map Features:

- Interactive panel: familiar Google Maps panel with dynamic loading. Allows for easy manipulation of map display.
- Styling: basic dark-themed styling that matches our UI's general red/black theme. This directly affects the appearance of key elements such as roads, buildings, and bodies of water, and transit stations.
- Layout Overview: the panel takes up approximately 2/3 of the page width, and accounts for smaller devices with adaptive resizing (note: unsure if this was needed initially, but decided to include adaptive feature after seeing other projects during our class peer review. Adaptive resizing will need to be added to event card elements).
- Header Information: shows current search location in map header, which updates when typing in real time.
- Loading States: Initial loading text that dissolves when map is ready. This informs users that the service is running as intended.

Location & Navigation Features:

- Dynamic Centering: the map automatically centers on the user's submitted location, with the latitude/longitude coordinates being converted from

address/city strings via Google's Geocoding service (part of the libraries offered for Maps JavaScript API).
- Default Location: falls back to Vancouver, BC as default center if no location provided.
- Zoom Control: upon new location submit, the map automatically zooms to an appropriate level (12x, technically "city-view").
- Real-time Location Updates: the map only updates after the user hits "Enter" within the header, shifting the view to their newly provided location. This prevents needless calls to the Geocoding service (which was a problem previously encountered).

Event Markers:

- Dynamic Event Markers: creates markers for all events with venue coordinates (currently uses a temporary blue marker). Each marker also is clickable, and shows a small info window of key event details (e.g., title, venue name, date & time, price, etc...).
- Marker Management: upon starting a new event search (i.e., clicking "Back to Search"), old markers are removed off the map and cleared from memory.
- Coordinate Extraction: location coordinates are extracted from returned Ticketmaster API data for accurate placement of markers.

Google Routes API:

Core Route Functionality

- **Interactive Route Calculation:** Integrates the Google Routes API for highly accurate calculation and display of routes between the user's location and their event venue.
- **Styling:** Features clean, dark-themed polyline styling that aligns with the app's user interface.
- **Layout Overview:** Routing options are displayed selectively. After a user chooses an event and clicks "Route," the content panel shows the available routing options.
- **Loading States:** A "Calculating Route..." text appears and dissolves on the calculation button to inform users that the service is working as intended.
- **External Google Maps Compatibility:** An optional "Open in Google Maps" button lets users export their planned route. This is ideal for those who prefer the more detailed options available in the main Google Maps application.

Location & Route Features

- **Dynamic Route Calculation:** The system automatically calculates the trip between the user's location and the event venue. It can do this in two ways:
    - **Address String:** Converts a typed address into latitude/longitude coordinates using Google's Geocoding service.
    - **"Use My Location" Button:** Leverages the browser's built-in GPS functionality to get the user's coordinates directly. This is a great alternative if users are unsure of their current address, though accuracy can vary.
- **Travel Mode Control:** Users can select from four travel modes: **walking**, **driving**, **transit**, or **biking**. The map and route polyline update to reflect the chosen mode.
- **Real-time Route Updates:** The route only updates after the user clicks "Calculate Route & Directions." This allows users to compare different travel options and see the route's complexity, time, and distance before finalizing.

Route Visualization & Polyline Management

- **Optimal Route Selection:** By default, all routes are calculated using Google's `TRAFFIC_AWARE_OPTIMAL` setting. This ensures the API returns the best possible route based on current traffic conditions and optimal travel time.
- **Polyline Clearing:** When a new route calculation is initiated (e.g., by changing the travel mode or location), the old polyline is removed from the map and cleared from memory to prevent visual clutter.
- **Coordinate Processing:** Route coordinates are extracted from the returned Routes API data. They are then processed into an array of coordinate objects that are plotted on the map panel to draw the visual route.
- **Turn-by-Turn Navigation Instructions:** these are available per event card, and provide several navigation options in transit modes between user start and end points. Also, the route can be optionally viewed in Google Maps.

Description of CI/CD pipeline

Our application uses both GitHub Actions and Render’s auto-deploy feature as its CI/CD pipeline. For continuous integration, GitHub Actions uses a .yml file to

automatically run tests on every push and pull request to the repository. For continuous deployment, we use Render's automatic deployment, which waits for CI checks to pass, then builds and runs automatically once changes are made. This pipeline is used to ensure that the basic functions of our application are still working before deploying it, which allows us to easily make changes without having to worry if anything has completely broken.

Step-by-step overview of CI/CD:

Continuous Integration - GitHub Actions

1. Triggers on:
    - Push to main branch
    - Pull request to main branch
    - Ensures changes are validated before merging to main
2. Job initialization
    - Creates new Ubuntu Linux VM
3. Checkout code
    - Copies code from repository to VM
4. Setup Node.js
    - Installs v22 of Node.js
    - Configures npm caching
    - Subsequent runs are much faster due to dependency caching
5. Install dependencies
    - Installs all dependencies using `ci` (clean install)
    - `--legacy-peer-deps` handles potential version conflicts
6. Run tests
    - Runs frontend test suites
    - Uses Vitest, which runs faster than Jest
7. Build
    - Builds after all tests have passed
8. Monitoring
    - GitHub sends notifications through email if any CI checks fail

Continuous Deployment - Render

1. Triggers on:
    - Changes to code in main branch
    - After CI checks have passed
    - Ensures application is functioning before deploying
2. Build and deploy
    - Installs dependencies and builds the app before deploy

3. Monitoring
    - Render sends email notifications if the build or deploy step has failed
        - Also sends the commit logs that caused the failure
    - Render dashboard displays response times and uptime

Description of the project's testing strategy and how it was implemented:

As previously mentioned, our CI/CD is used to ensure basic functionality. This is so that once changes are made to the main branch of the repository, we can ensure the most important features of our website are functional before deployment. However, certain features are difficult to test for, such as the map UI. These features currently must be tested manually. Our test functions were also implemented relatively late into development, so it's likely that they were not utilized fully up until the deadline, however, they would still be useful for further development.

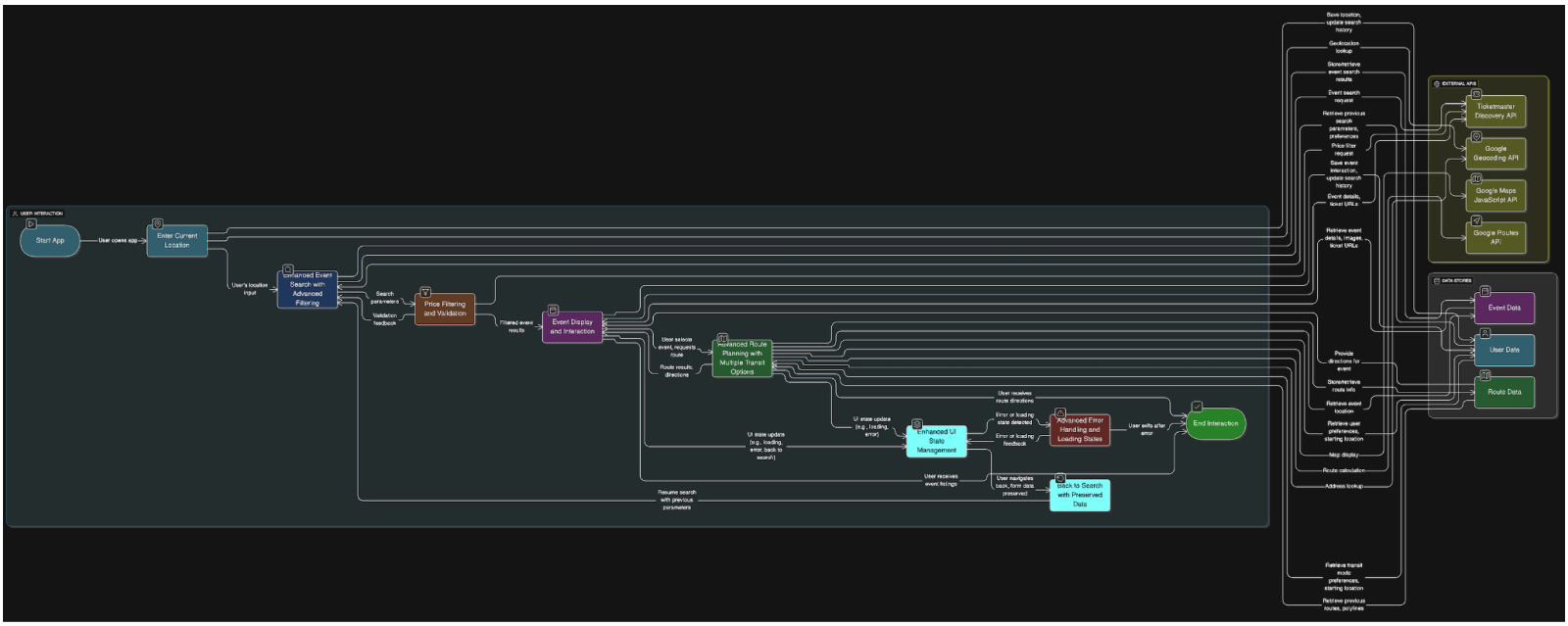
For our test functions, we used the React testing library and Vitest due its integration with Vite and faster execution times than Jest. Vitest includes a "mocking" feature, which allows for the testing of certain components without the influence of any external factors or dependencies. It also allows for faster execution since there is no need for external resources. These mocks are used in all of our tests.

Unit tests include checking the functionality of individual UI components, such as filling in text fields and pressing buttons. Subsequent tests integrate multiple of these unit tests to test features seen later in the flow of the application, for example, submitting a location, then filling out multiple filters, and lastly checking event details. Some tests use mock data to see if the application handles certain data correctly and returns the right results. In future testing implementation, I would like to fully automate the testing of the main features, as well as include more potential cases like edge cases.

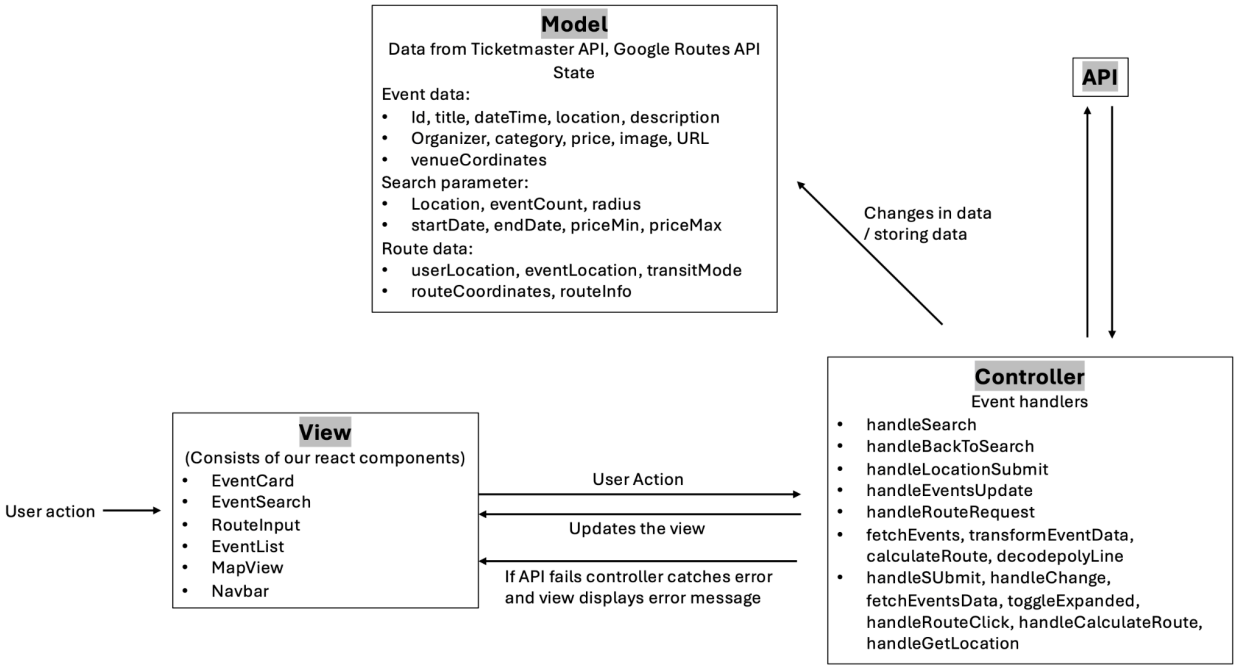
Detailed diagram of the project's architecture (updated level1 DFD of application's structure and how data flows within the application) (updated MVC model of application):

Updated Data Flow Diagram

Link to image:
https://github.com/CMPT-276-SUMMER-2025/final-project-17-sunsets/blob/main/docs/design/dfds.png



Updated Model View Controller Diagram



Model: The model is what manages all of the data that runs our web application. For instance as supported in milestone 1 we said that the model was responsible for storing

the lists of events, keeping track of particular routes and storing the state. Moreover, we figured that we could categorize our model into three sections as shown below which shows how our current updated model links to our web application:-

- Event Model - For this model we see that every event that is pulled from the Ticketmaster API will be saved in a specific structure, for instance these will consist of a title, ID, date and time, location, description, organizer, category, price, URL, image and the coordinates.
- Search Parameter Model - For this users will enter in specific locations and for these locations a set of search parameters will be required to be filled by the user. For instance search preferences such as the filters for radius from specific location, price range, and date range. These parameters are stored in state so that searching and sorting from the API can keep going on.
- Route Data Model - For this web application information regarding a users location, an event location, a selected transport mean and event route coordinates should be kept in track of. Hence the reason for this model.

We kept these models up to date by using useState hooks in the App.jsx file and with the help of the controller specific functions.

Controller: The process that links the users actions to a particular data change is what the controller is in charge of. For instance, according to our updated MVC diagram we can see that the controller will manage multiple steps of interactions with the Ticketmaster and Google Routes API. Furthermore the state transitions within our web application is part of the controller. We use the event handlers to implement these state transitions, API interactions and the users actions within our specified components. For instance, within MeropoLive the controllers identified would be:-

- Within App.jsx - handleSearch(), handleLocationSubmit(), handleRouteRequest() and handleBackToSearch()
- Controllers for API
    - Within ticketmasterApi.js - fetchEvents(), transformEventData()
    - Within routesApi.js - calculateRoute(), decodePolyline()
- Controller within the Component - EventSearch, EventList, EventCard, and RouteInput.

View: The components that we built using React.js is what makes up the view.
The components consists of:-

- Navbar - Contains logo, title and the input for a specified location
- EventSearch - A form that has a category and date,radius, price filters.
- EventList and EventCard - this is to help see the search results.
- Mapview - This displays our map with markers when events are chosen.

- RouteInput - This is for the user to enter a mode of transport and a location to start from.

List of known bugs and issues with the project and their severity (table format) (Bug reports should include description of the bug, steps to reproduce, and severity level and link to Github issue):

Issues regarding event cards may not be reproduced exactly the same depending on when the bug is reproduced as the events are relative to the current date of the search.

Bug	Steps to reproduce	Severity	State	Link to issue
Back to search button: Clicking back to search button results in an empty view instead of back to search preferences.	(Bug fixed)	High: Needs to work as it is an important feature of the application	Resolved	https://github.com/CMPT-276-SUMMER-2025/final-project-17-sunsets/issues/45
Enter button in navbar: Doesn't refresh events with new location search. Shows previously searched events.	(Bug fixed)	Medium: While it is a core feature, refreshing the page would fix this issue. Also, most users would search a location once.	Resolved	https://github.com/CMPT-276-SUMMER-2025/final-project-17-sunsets/issues/26
Events outside price range: Events outside of filtered price range being shown.	- Input location “Coquitlam”. - Input 0 in the price min and price max filter and search. - Find “ZZ Top”. This event is a none free event.	Medium: A part of the application is the ability to search by price of event but the bug doesn't make the application unusable.	Unresolved	https://github.com/CMPT-276-SUMMER-2025/final-project-17-sunsets/issues/44

Events with wrong category: Events with categories conflicting the search filter.	- Input location “Vancouver”. - Put “Sports” in the category filter and search. - Scroll down and find “keshi: REQUIEM TOUR”. This event is in the Music category but appears when searching for sport events.	Low: This only happens when a key word is found on the listing on the event. Such as having the word “sports” in the location name being shown when searched for “sports” category	Unresolved	https://github.com/CMPT-276-SUMMER-2025/final-project-17-sunsets/issues/51
Can’t find route when event doesn't retrieve address: Unable to properly retrieve address of event causing routing function to not find proper route.	- Input location “Surrey”. - Search. - Find “Chessington World of Adventure Resort” - Click on “Route”. - Enter “1260 Riverwood Gate, Port Coquitlam” as the starting location. - Click “Calculate Route & Directions”. This will result in a “No route found” error because the google maps API uses the closest match to the “undefined”	Medium: Happens with a few events when address is retrieved as “undefined”. Google maps will select the best match. Not a common bug.	Unresolved	https://github.com/CMPT-276-SUMMER-2025/final-project-17-sunsets/issues/52

	address.			

Description of the project's future work and potential improvements (Any features that were not implemented and how they could be in the future):

- We could have implemented a minimalistic landing page with only the title of our site with a search bar for the user to enter the location directly which would later transition into our current web-page with the search preference event card showing up with the map view as default. This could be useful for users as it would not confuse users who might have not watched a demo of how to use the website.
- If we implemented the landing page we could have an option for either first time users in which when clicked there could be step-by-step assistance. For instance arrows pointing to location on the web page with a short but clear description of what the user needs to do.
- When the events are found it would have been great if we could have had a feature where when any event is clicked the map highlights that specific location in a different colour or even by only showing that events location rather than with all the other events.
- A potential improvement we could have made would have been to use an AI API to do the booking automatically for the user. For instance, we could have the user have the option to choose to do it manually or through an AI assistant. The advantage of the AI assistant would be ideal for users who have a clear understanding of which event they want to go to, the timing and all important details. For example, we could have a text box in which the user could type in all the details of what they are in search of and the AI assistant uses the preferences and searches to make a booking for the user. (we could also make use of the AI assistant could also use the preferences to fill in fields within our website for the user)
- According to our peer-review, for users who are visually impaired we could have first displayed the search preferences without showing the map view so that users could see the side bar better and clearly, or added a zoom in function which would have minimized the mapview and expanded the event card for displaying the search preferences and once all preferences are selected the map automatically expands to its default view so that the user could see all the locations of the events and the route for a desired event.

Lessons learned and project takeaways

Matching schedules for meetings: Everyone's schedules seem to clash so finding time to voice call about the project was a challenge. We realized that near the end of our project when all the details were in place, it was not beneficial for us to have meetings anymore. The meetings also cut into time which could be spent on the project itself. Instead, to overcome the problem of schedule conflicts, we focused more on constant communication. This helped save time and stress while also allowing us to keep steady progress on our project. This decision made everything more efficient.

Project roles: Initially, we assigned ourselves arbitrary roles as we weren't sure what to expect or what our strengths might be in terms of development. It was also somewhat difficult to assign certain tasks to people because of this. But, over time we seemed to fall into our true roles and began to understand what our strengths were. We also began to better understand the use of Kanban and how it could be used to better assign tasks.

Contributions to the project:

- **Tyrese:** dev contributions: implementation of backend API logic and basic frontend application structure. Specific to M2 report: detailed descriptions of API features; DFD diagram (still working on this).
- **Yuginda:** Was in charge of stylings of the web application, designed the logo and helped with basic structure of the web app. Was responsible for updated mvc and description for projects future work and potential improvements.
- **Sumin:** Project manager. Created the outlines for all reports and documents. Oversaw progress and due dates. Set reminders and organized the project into tasks which were then delegated. Kept everyone on track. Reviewed the project and provided feedback where needed. Designed and drew favicon.
- **Jacob:** Created initial outline of project schedule and tasks. Constructed CI/CD pipeline, including .yml file and setting up Render. Created automated test functions. Responsible for editing video presentations.

Changelog table with revisions since previous milestones: (N/A: no changes)

Milestone	Past state	Revision
Milestone 0	Chosen APIs are Eventbrite and Google maps.	Eventbrite API is replaced with Ticketmaster API.
Milestone 1	N/A	N/A
Milestone 1.5	N/A	N/A

Peer testing feedback form (survey questions):
General Impressions, Task-Specific Feedback, Usability Suggestions.

Peer testing feedback received from classmates (survey result):
General Impressions:

- Looks clean, good/convenient UI design.
- The website is intuitive to use and pleasing to look at (elements are minimal and good use of color). The website works as intended.
- The website is very straight forward to use, no confusion. I personally enjoy the dark color scheme. Error handling for invalid input is thorough.
- UI is intuitive, easy to navigate and esthetically pleasing.
- I appreciate the window of time to select for events. All events were listed as “Event organizer”, I could not visit any site to purchase a ticket.

Task-Specific Feedback:

- Tasks were good and the place holders helped a lot.
- Easy to understand what to do and output to expect.
- Tasks are easy to follow and complete. Features that don’t work are clearly highlighted.
- I wish I could expand the side bar or zoom in, I am slightly visibly impaired.
- #3a, could not visit the site to purchase a ticket.

Usability Suggestions:

- No suggestions.
- Generally, everything on the website is easy to use. No suggestions. Very nice final product?
- The right grid does not line up at the bottom with the map section. Logo for the website would be nice to include. Some listings are missing a button to the ticketmaster site - likely because it's too late to buy tickets. It would be nice to clarify the reason.
- Maybe other metrics like miles or other. Currently based on location selection. For route planning chosen mode of transportation and time.
- The Google Map provided has pins to their global paid sites which do not relate to the events, this could be quite confusing.